13.4 会话裁剪（Session Pruning）

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~250,000 tokens，输出 ~18,000 tokens（本章合计）

随着对话的进行，会话中累积的消息会越来越多。对于大语言模型（LLM）来说，每次调用都需要将完整的对话历史作为输入，而模型的上下文窗口有固定的 Token 上限。OpenClaw 通过**上下文裁剪**（Context Pruning）和**上下文窗口守卫**（Context Window Guard）两套机制来管理这个问题。

> **衍生解释**：上下文窗口（Context Window）是 LLM 的一个核心约束。每个模型都有一个 Token 上限（例如 Claude 的 200K tokens、GPT-4o 的 128K tokens），所有的系统提示词、对话历史和工具结果都必须在这个窗口内。Token 可以粗略理解为"子词"（sub-word），英文中大约每 4 个字符为 1 个 token，中文中每个汉字约 1-2 个 token。当对话历史超出窗口大小时，LLM 会拒绝处理，因此必须在调用前裁剪多余的内容。

13.4.1 工具结果裁剪：在 LLM 调用前修剪旧工具结果

Agent 工作过程中，工具调用的结果（如 bash 命令的输出、文件内容、搜索结果等）往往占据上下文的大量空间。OpenClaw 在 `src/agents/pi-extensions/context-pruning/` 目录下实现了一套两阶段裁剪策略：**软裁剪**（Soft Trim）和**硬清除**（Hard Clear）。

裁剪配置

`src/agents/pi-extensions/context-pruning/settings.ts` 定义了裁剪的配置结构和默认值：

```ts
// src/agents/pi-extensions/context-pruning/settings.ts
export const DEFAULT_CONTEXT_PRUNING_SETTINGS: EffectiveContextPruningSettings = {
  mode: "cache-ttl",
  ttlMs: 5 * 60 * 1000,          // 5 分钟缓存 TTL
  keepLastAssistants: 3,          // 保护最近 3 轮 Assistant 回复
  softTrimRatio: 0.3,             // 上下文使用率 > 30% 时触发软裁剪
  hardClearRatio: 0.5,            // 上下文使用率 > 50% 时触发硬清除
  minPrunableToolChars: 50_000,   // 可裁剪工具结果至少 50K 字符才执行硬清除
  tools: {},                       // 工具白名单/黑名单
  softTrim: {
    maxChars: 4_000,              // 超过 4K 字符的工具结果才触发软裁剪
    headChars: 1_500,             // 保留头部 1500 字符
    tailChars: 1_500,             // 保留尾部 1500 字符
  },
  hardClear: {
    enabled: true,
    placeholder: "[Old tool result content cleared]",
  },
};
```

`mode` 目前只支持 `"cache-ttl"` 模式：裁剪结果会被缓存，在 TTL（默认 5 分钟）过期前不会重复计算。这避免了每次 LLM 调用都重新执行裁剪逻辑。

裁剪触发机制

裁剪以 Pi Agent 的扩展（Extension）形式注入到 Agent 运行时中。每次 LLM 调用前，扩展会检查是否需要裁剪：

```
// src/agents/pi-extensions/context-pruning/extension.ts
export default function contextPruningExtension(api: ExtensionAPI): void {
  api.on("context", (event: ContextEvent, ctx: ExtensionContext) => {
    const runtime = getContextPruningRuntime(ctx.sessionManager);
    if (!runtime) return undefined;

    // cache-ttl 模式：TTL 未过期则跳过
    if (runtime.settings.mode === "cache-ttl") {
      const lastTouch = runtime.lastCacheTouchAt ?? null;
      if (!lastTouch || Date.now() - lastTouch < runtime.settings.ttlMs) {
        return undefined;
      }
    }

    const next = pruneContextMessages({
      messages: event.messages,
      settings: runtime.settings,
      ctx,
    });

    if (next === event.messages) return undefined;  // 未发生裁剪
    runtime.lastCacheTouchAt = Date.now();          // 更新缓存时间戳
    return { messages: next };                      // 返回裁剪后的消息列表
  });
}
```

> **衍生解释**：这里使用了事件驱动的扩展模式。Pi Agent Core 在每次准备 LLM 调用时会触发 `context` 事件，扩展可以拦截这个事件并修改即将发送给 LLM 的消息列表。这种设计使得裁剪逻辑与 Agent 核心完全解耦——可以独立配置、启用或禁用。

两阶段裁剪算法

`pruneContextMessages()` 是裁剪的核心算法，位于 `src/agents/pi-extensions/context-pruning/pruner.ts`：

```typescript
// src/agents/pi-extensions/context-pruning/pruner.ts
const CHARS_PER_TOKEN_ESTIMATE = 4;  // 粗略换算：1 token ≈ 4 字符

export function pruneContextMessages(params: {
  messages: AgentMessage[];
  settings: EffectiveContextPruningSettings;
  ctx: Pick<ExtensionContext, "model">;
}): AgentMessage[] {
  const { messages, settings } = params;
  const contextWindowTokens = ctx.model?.contextWindow;
  if (!contextWindowTokens) return messages;

  const charWindow = contextWindowTokens * CHARS_PER_TOKEN_ESTIMATE;

  // 1. 确定保护区域：最近 N 轮 Assistant 回复不可裁剪
  const cutoffIndex = findAssistantCutoffIndex(messages, settings.keepLastAssistants);
  if (cutoffIndex === null) return messages;

  // 2. 安全边界：第一条 user 消息之前的内容不可裁剪
  //    （保护 SOUL.md 等引导文件的读取结果）
  const firstUserIndex = findFirstUserIndex(messages);
  const pruneStartIndex = firstUserIndex ?? messages.length;

  let totalChars = estimateContextChars(messages);
  let ratio = totalChars / charWindow;

  // 如果使用率低于软裁剪阈值，无需裁剪
  if (ratio < settings.softTrimRatio) return messages;

  // 收集可裁剪的工具结果索引
  const prunableToolIndexes: number[] = [];
  // ...
}
```

算法的关键在于两个保护区域：

1. **尾部保护**：最近的 `keepLastAssistants`（默认 3）轮 Assistant 回复及其关联的工具结果不可裁剪，因为它们是当前对话的直接上下文
2. **头部保护**：第一条 `user` 消息之前的所有内容不可裁剪，因为这些通常是引导文件（如 `SOUL.md`、`AGENTS.md`）的读取结果，对 Agent 的行为至关重要

两个保护区域之间的工具结果消息，按照以下流程处理：

阶段	触发条件	操作	效果
阶段 1：软裁剪	`ratio > softTrimRatio` （默认 0.3）	保留头部 1500 + 尾部 1500 字符，中间用 `...` 替代	保留工具结果的关键信息
阶段 2：硬清除	`ratio > hardClearRatio` （默认 0.5）	整个工具结果替换为占位符文本	彻底释放空间

软裁剪的实现：

```
// src/agents/pi-extensions/context-pruning/pruner.ts
function softTrimToolResultMessage(params: {
  msg: ToolResultMessage;
  settings: EffectiveContextPruningSettings;
}): ToolResultMessage | null {
  const { msg, settings } = params;
  // 图片工具结果不裁剪（难以安全地部分裁剪）
  if (hasImageBlocks(msg.content)) return null;

  const parts = collectTextSegments(msg.content);
  const rawLen = estimateJoinedTextLength(parts);
  if (rawLen <= settings.softTrim.maxChars) return null;  // 不够长，跳过

  // 保留头尾，中间截断
  const head = takeHeadFromJoinedText(parts, settings.softTrim.headChars);
  const tail = takeTailFromJoinedText(parts, settings.softTrim.tailChars);
  const trimmed = `${head}\n...\n${tail}`;
  const note = `\n\n[Tool result trimmed: kept first ${headChars} chars ` +
    `and last ${tailChars} chars of ${rawLen} chars.]`;
  return { ...msg, content: [asText(trimmed + note)] };
}
```

硬清除更为激进——直接用占位符替换整个工具结果：

```
// src/agents/pi-extensions/context-pruning/pruner.ts（硬清除片段）
for (const i of prunableToolIndexes) {
  if (ratio < settings.hardClearRatio) break;  // 已降到阈值以下
  const msg = outputAfterSoftTrim[i];
  // 替换为占位符
  const cleared = { ...msg, content: [asText(settings.hardClear.placeholder)] };
  next[i] = cleared;
  // 重新计算使用率
  totalChars += estimateMessageChars(cleared) - estimateMessageChars(msg);
  ratio = totalChars / charWindow;
}
```

工具结果完整性守卫

除了裁剪，OpenClaw 还需要保证会话转录中工具调用和结果的**配对完整性**。某些 LLM 提供者（如 Anthropic Claude）要求每个工具调用（ `toolCall` ）都必须有对应的工具结果（ `toolResult` ），否则会拒绝请求。

`src/agents/session-tool-result-guard.ts` 通过猴子补丁（Monkey-Patch） `SessionManager.appendMessage` 方法来追踪未完成的工具调用：

```ts
// src/agents/session-tool-result-guard.ts
export function installSessionToolResultGuard(sessionManager: SessionManager) {
  const originalAppend = sessionManager.appendMessage.bind(sessionManager);
  const pending = new Map<string, string | undefined>();  // toolCallId → toolName

  const flushPendingToolResults = () => {
    // 为所有未收到结果的工具调用生成合成结果
    for (const [id, name] of pending.entries()) {
      const synthetic = makeMissingToolResult({ toolCallId: id, toolName: name });
      originalAppend(synthetic);
    }
    pending.clear();
  };

  sessionManager.appendMessage = (message: AgentMessage) => {
    if (message.role === "assistant") {
      // 提取工具调用 ID，加入待处理集合
      const toolCalls = extractAssistantToolCalls(message);
      if (pending.size > 0 && toolCalls.length === 0) flushPendingToolResults();
      for (const call of toolCalls) pending.set(call.id, call.name);
    }
    if (message.role === "toolResult") {
      // 收到工具结果，从待处理中移除
      const id = extractToolResultId(message);
      if (id) pending.delete(id);
    }
    return originalAppend(message);
  };

  return { flushPendingToolResults, getPendingIds: () => Array.from(pending.keys()) };
}
```

> **衍生解释**：猴子补丁（Monkey-Patch）是一种在运行时替换对象方法的技术。这里通过替换 `appendMessage` 方法，在不修改 Pi Agent Core 源码的情况下加入了工具结果追踪逻辑。虽然这种技术在生产代码中应谨慎使用，但在需要拦截第三方库行为时是一种实用的方案。

13.4.2 上下文窗口守卫（Context Window Guard）

裁剪机制处理的是"如何缩减上下文"，而上下文窗口守卫处理的是"上下文窗口本身是否合理"。 `src/agents/context-window-guard.ts` 提供了上下文窗口大小的解析和校验。

上下文窗口大小解析

不同的模型有不同的上下文窗口大小，用户还可能通过配置文件覆盖这个值。 `resolveContextWindowInfo()` 按优先级确定最终的窗口大小：

```
// src/agents/context-window-guard.ts
export type ContextWindowSource = "model" | "modelsConfig" | "agentContextTokens" | "default";

export function resolveContextWindowInfo(params: {
  cfg: OpenClawConfig | undefined;
  provider: string;
  modelId: string;
  modelContextWindow?: number;     // 模型元数据中的窗口大小
  defaultTokens: number;
}): ContextWindowInfo {
  // 优先级 1：用户在 models.providers 中手动配置的值
  const fromModelsConfig = /* cfg.models.providers[provider].models[modelId].contextWindow */;
  // 优先级 2：模型自身报告的值
  const fromModel = normalizePositiveInt(params.modelContextWindow);
  // 优先级 3：默认值
  const baseInfo = fromModelsConfig
    ? { tokens: fromModelsConfig, source: "modelsConfig" }
    : fromModel
      ? { tokens: fromModel, source: "model" }
      : { tokens: Math.floor(params.defaultTokens), source: "default" };

  // Agent 级别的硬上限：如果配置了 contextTokens 且小于基础值，则使用它
  const capTokens = normalizePositiveInt(params.cfg?.agents?.defaults?.contextTokens);
  if (capTokens && capTokens < baseInfo.tokens) {
    return { tokens: capTokens, source: "agentContextTokens" };
  }
  return baseInfo;
}
```

安全阈值检查

确定窗口大小后， `evaluateContextWindowGuard()` 检查是否低于安全阈值：

```
// src/agents/context-window-guard.ts
export const CONTEXT_WINDOW_HARD_MIN_TOKENS = 16_000;   // 硬性最小值：16K
export const CONTEXT_WINDOW_WARN_BELOW_TOKENS = 32_000; // 警告阈值：32K

export function evaluateContextWindowGuard(params: {
  info: ContextWindowInfo;
}): ContextWindowGuardResult {
  const warnBelow = CONTEXT_WINDOW_WARN_BELOW_TOKENS;
  const hardMin = CONTEXT_WINDOW_HARD_MIN_TOKENS;
  const tokens = params.info.tokens;
  return {
    ...params.info,
    tokens,
    shouldWarn: tokens > 0 && tokens < warnBelow,   // 低于 32K：发出警告
    shouldBlock: tokens > 0 && tokens < hardMin,     // 低于 16K：拒绝运行
  };
}
```

当 `shouldBlock` 为 `true` 时，Gateway 会拒绝启动 Agent，因为在 16K tokens 以下，系统提示词、工具定义和最基本的对话历史都可能放不下，继续运行只会产生低质量的回复。

本节小结

OpenClaw 的会话裁剪体系由三个互补的机制组成：

1. **两阶段上下文裁剪**：软裁剪（保留头尾、截断中间）和硬清除（替换为占位符）的渐进策略，在控制上下文大小的同时尽量保留有价值的信息
2. **智能保护区域**：最近 N 轮回复和引导文件永远不被裁剪，确保 Agent 的当前对话上下文和基础人格不受影响
3. **工具结果完整性守卫**：通过追踪工具调用和结果的配对关系，自动为缺失的结果生成合成回复，满足 LLM 提供者的严格要求
4. **上下文窗口守卫**：在 Agent 启动前校验模型的上下文窗口大小，低于 16K tokens 时拒绝运行，低于 32K tokens 时发出警告

这套机制使得 OpenClaw 能够在长时间的对话中保持稳定运行，不会因为上下文溢出而崩溃或产生低质量回复。